

Отчёта по лабораторной работе 6

Имитационное моделирование

Нджову Нелиа

Содержание

Список иллюстраций

Список таблиц

1 Цель работы

Цель этой лабораторной работы - Реализация основных моделей в подходе сетей Петри.

2 Задание

1. Основная программа
2. Базовый прогон модели
3. Коэффициент заражения β
4. Анимация детерминированной динамики
5. Итоговый отчёт

3 Теоретическое введение

Реализована модель эпидемии SIR (Susceptible–Infectious–Recovered) с использованием сетей Петри. Модель описывает переходы между тремя состояниями:

- S (восприимчивые) — могут заразиться;
- I (инфицированные) — заражают других и выздоравливают;
- R (выздоровевшие / с иммунитетом) — больше не участвуют в эпидемии.

Сеть Петри содержит два перехода:

- infection: $S + I \rightarrow I + I$ (скорость β);
- recovery: $I \rightarrow R$ (скорость γ).

4 Выполнение лабораторной работы

4.1 1. Основная программа

Я создала файл `src/SIRPetri.jl` и скопировала заданный скрипт в него. Скрипт реализует вычислительную логику модели. Скрипт содержатся несколько основные функции. Функция `build_sir_network` для построения сети Петри, функция `simulate_deterministic` для детерминированной симуляции, функция `simulate_stochastic` для стохастической симуляции, функция `sir_ode` для вспомогательной функции ОДУ, функция для визуализации и функция для визуализации сети Петри. Я запускала и все работала хорошо(рис.1)


```

SIRPetri.jl ×
module SIRPetri
using AlgebraicPetri
using Catlab.CategoricalAlgebra
using Catlab.Graphics
using OrdinaryDiffEq
using Plots
using DataFrames
using Random

export build_sir_network, simulate_deterministic,
       simulate_stochastic
export plot_sir, to_graphviz_sir

"""
build_sir_network(β=0.3, γ=0.1)
Создаёт размеченную сеть Петри для модели SIR.
Возвращает (net::LabelledPetriNet, u0::Vector{Float64},
           states::Vector{Symbol})
"""
function build_sir_network(β = 0.3, γ = 0.1)
    states = [:S, :I, :R]
    # Создаём сеть Петри, описывая переходы напрямую
    # Переход infection: S + I → I + I
    # Переход recovery: I → R
    net = LabelledPetriNet(
        states,
        :infection => ([:S, :I] => [:I, :I]),
        :recovery => ([:I] => [:R]),
    )

    # Начальная маркировка: S=990, I=10, R=0
    u0 = [990.0, 10.0, 0.0]

    return net, u0, states
end

```

Рисунок 4.1: Основная программа

4.2 2. Базовый прогон модели

Я создала файл `scripts/sirpetri_run.jl` и скопировала заданный скрипт в него. Скрипт выполняет один базовый эксперимент с фиксированными параметрами $\beta = 0.3$, $\gamma = 0.1$, запуская два типа симуляции: детерминированную (решение ОДУ) и стохастическую (алгоритм Гиллеспи). Я преобразовала скрипт в литературный стиль и запускала его (рис.2)

```

sirpetri_run.jl ×
## Базовый прогон модели

using DrWatson
@quickactivate "project"
using Random
include(scrdir("SIRPetri.jl"))
using .SIRPetri
using DataFrames, CSV, Plots

# Выполняет один базовый эксперимент с фиксированными параметрами  $\beta$ 
# = 0.3,  $\gamma$  = 0.1.

# Параметры
 $\beta$  = 0.3
 $\gamma$  = 0.1
tmax = 100.0

# Создаём сеть
net, u0, states = build_sir_network( $\beta$ ,  $\gamma$ )

# **Запускает два типа симуляции**
#
# детерминированную (решение ОДУ) – даёт плавную усреднённую
# динамику;
df_det = simulate_deterministic(net, u0, (0.0, tmax), saveat = 0.5,
rates = [ $\beta$ ,  $\gamma$ ])

#стохастическую (алгоритм Гиллеспи) – учитывает случайные
# флуктуации.
Random.seed!(123)
df_stoch = simulate_stochastic(net, u0, (0.0, tmax), rates = [ $\beta$ ,  $\gamma$ ])

```

Рисунок 4.2: Скрипт базового эксперимента

После запуска, получила два графика. Детерминированный график показывает классический пик эпидемии: рост I максимум, затем спад до нуля; R растёт и выходит на плато, S падает. Стохастический график может иметь шумы и, возможно, немного отличаться по времени пика и амплитуде – это демонстрирует влияние случайности (рис 3 и 4)

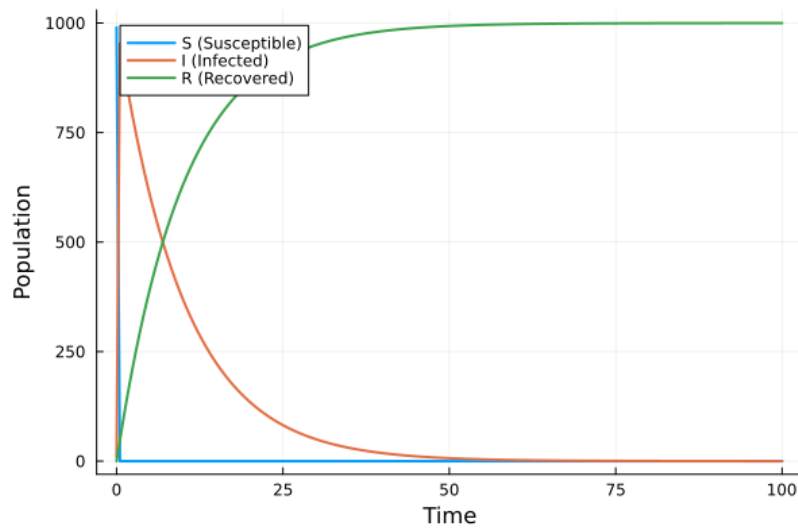


Рисунок 4.3: Детерминистический график динамики SIR

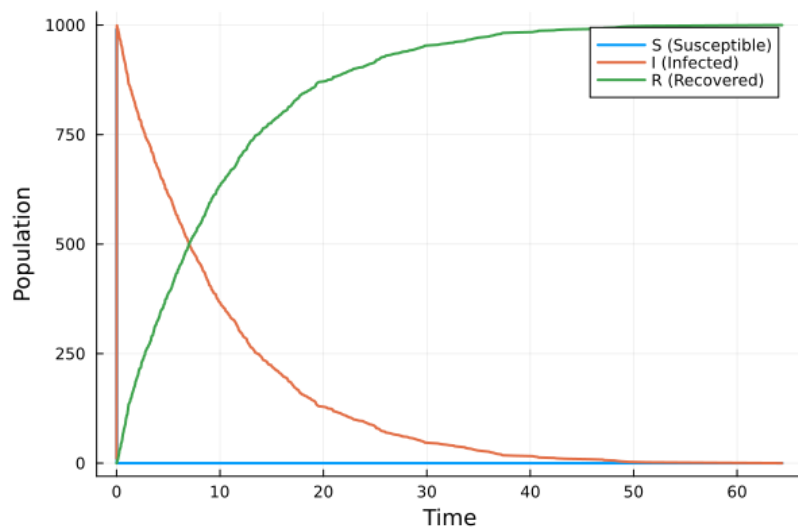
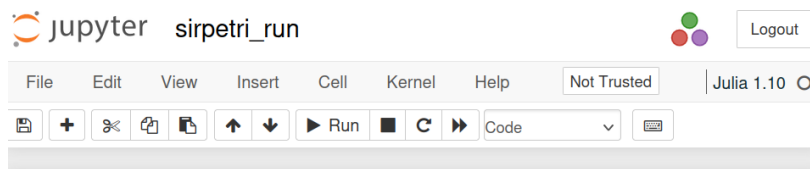


Рисунок 4.4: Стохастический график динамики SIR

Я генерировала файл jupyter, запускала все ячейке и все сработала без ошибок(рис.5)



Базовый прогон модели

```
In [1]: using DrWatson
@quickactivate "project"
using Random
include(sourcedir("SIRPetri.jl"))
using .SIRPetri
using DataFrames, CSV, Plots
```

Выполняет один базовый эксперимент с фиксированными параметрами $\beta = 0.3$, $\gamma = 0.1$.

Рисунок 4.5: файл jupyter

Ниже приведен код, результаты расчетов и графики для кода в файле `scripts/sirpetri_run.qmd`

4.3 Базовый прогон модели

```
using DrWatson
@quickactivate "project"
using Random
include(sourcedir("SIRPetri.jl"))
using .SIRPetri
using DataFrames, CSV, Plots
```

```
└ Warning: DrWatson could not find find a project file by recursively checking given `dir`
  | (given dir: /home/nelianjovu)
```

```
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

```
WARNING: replacing module SIRPetri.
```

```
WARNING: using SIRPetri.build_sir_network in module Main conflicts with an existing iden
```

WARNING: using SIRPetri.simulate_deterministic in module Main conflicts with an existing
WARNING: using SIRPetri.simulate_stochastic in module Main conflicts with an existing id
WARNING: using SIRPetri.plot_sir in module Main conflicts with an existing identifier.

Выполняет один базовый эксперимент с фиксированными параметрами β
 $= 0.3, \gamma = 0.1$.

Параметры

```
 $\beta$  = 0.3  
 $\gamma$  = 0.1  
tmax = 100.0
```

100.0

Создаём сеть

```
net, u0, states = build_sir_network( $\beta$ ,  $\gamma$ )
```

(AlgebraicPetri.LabelledPetriNet:

T = 1:2

S = 1:3

I = 1:3

O = 1:3

Name = 1:0

it : I $\rightarrow$ T = [1, 1, 2]

is : I $\rightarrow$ S = [1, 2, 2]

ot : O $\rightarrow$ T = [1, 1, 2]

os : O $\rightarrow$ S = [2, 2, 3]

tname : T $\rightarrow$ Name = [:infection, :recovery]

sname : S $\rightarrow$ Name = [:S, :I, :R], [990.0, 10.0, 0.0], [:S, :I, :R])

Запускает два типа симуляции

детерминированную (решение ОДУ) — даёт плавную усреднённую динамику;

```
df_det = simulate_deterministic(net, u0, (0.0, tmax), saveat = 0.5, rates = [□, □])
```

	time	S	I	R
	Float64	Float64	Float64	Float64
1	0.0	990.0	10.0	0.0
2	0.5	2.47053e-7	952.691	47.3086
3	1.0	3.28252e-7	906.228	93.7719
4	1.5	-1.78365e-7	862.031	137.969
5	2.0	-2.6597e-7	819.989	180.011
6	2.5	2.389e-7	779.998	220.002
7	3.0	-3.23518e-7	741.957	258.043
8	3.5	-2.30682e-7	705.771	294.229
9	4.0	3.04028e-8	671.35	328.65
10	4.5	3.63345e-8	638.608	361.392
11	5.0	-3.54734e-7	607.463	392.537
12	5.5	-3.48967e-8	577.837	422.163
13	6.0	2.30028e-7	549.655	450.345
14	6.5	3.24586e-8	522.848	477.152
15	7.0	-3.61058e-7	497.349	502.651
16	7.5	-2.03624e-7	473.093	526.907
17	8.0	4.39154e-7	450.02	549.98
18	8.5	1.58379e-7	428.072	571.928
19	9.0	-2.08057e-7	407.195	592.805
20	9.5	-2.44636e-7	387.335	612.665
21	10.0	1.2209e-7	368.445	631.555
22	10.5	1.73937e-7	350.476	649.524
23	11.0	6.83511e-8	333.383	666.617
24	11.5	-1.72158e-7	317.123	682.877
25	12.0	3.64045e-7	301.657	698.343
26	12.5	-3.92833e-7	286.945	713.055
27	13.0	-3.79316e-7	272.951	727.049
28	13.5	1.90957e-7	259.639	740.361
29	14.0	-2.2568e-7	246.976	753.024
30	14.5	4.2444e-7	234.931	765.069
...	...	...	...	...

стохастическую (алгоритм Гиллеспи) — учитывает случайные флуктуации.

```
Random.seed!(123)  
df_stoch = simulate_stochastic(net, u0, (0.0, tmax), rates = [0, 0])
```


	time	S	I	R
	Float64	Float64	Float64	Float64
1	0.0	990.0	10.0	0.0
2	0.000219318	989.0	11.0	0.0
3	0.00025471	988.0	12.0	0.0
4	0.000435457	987.0	13.0	0.0
5	0.00124186	986.0	14.0	0.0
6	0.00137312	985.0	15.0	0.0
7	0.00151759	984.0	16.0	0.0
8	0.00219412	983.0	17.0	0.0
9	0.00239638	982.0	18.0	0.0
10	0.00245828	981.0	19.0	0.0
11	0.00253136	980.0	20.0	0.0
12	0.00274985	979.0	21.0	0.0
13	0.00277041	978.0	22.0	0.0
14	0.00291987	977.0	23.0	0.0
15	0.00327978	976.0	24.0	0.0
16	0.00338914	975.0	25.0	0.0
17	0.00360517	974.0	26.0	0.0
18	0.00367641	973.0	27.0	0.0
19	0.00394413	972.0	28.0	0.0
20	0.00406046	971.0	29.0	0.0
21	0.00437152	970.0	30.0	0.0
22	0.00448011	969.0	31.0	0.0
23	0.00458821	968.0	32.0	0.0
24	0.00473125	967.0	33.0	0.0
25	0.00474643	966.0	34.0	0.0
26	0.00497684	965.0	35.0	0.0
27	0.0049977	964.0	36.0	0.0
28	0.00501235	963.0	37.0	0.0
29	0.00504887	962.0	38.0 ₁₇	0.0
30	0.00508054	961.0	39.0	0.0
...	...	...	...	...

Сохраняет результаты в CSV-файлы и строит графики $S(t)$, $I(t)$, $R(t)$ для обоих типов.

```
CSV.write(datadir("sir_det.csv"), df_det)
CSV.write(datadir("sir_stoch.csv"), df_stoch)
```

```
"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab06/project/data/sir_stoch.csv"
```

Графики

```
p_det = plot_sir(df_det)
savefig(plotsdir("sir_det_dynamics.png"))

p_stoch = plot_sir(df_stoch)
savefig(plotsdir("sir_stoch_dynamics.png"))

println("Базовый прогон завершён. Результаты в data/ и plots/")
```

Базовый прогон завершён. Результаты в data/ и plots/

Я добавила несколько параметров, чтобы посмотреть, как будет выглядеть график(рис.6)

```

sirpetri_run.jl × sirpetri_run_param.jl ×
# ## Базовый прогон модели SIR (Petri nets)
#
using DrWatson
@quickactivate "project"
using Random
include(scrdir("SIRPetri.jl"))
using .SIRPetri
using DataFrames, CSV, Plots

# Выполняет базовые эксперименты с разными параметрами  $\beta$  и  $\gamma$ .

# Параметры для трёх экспериментов
experiments = [
    ( $\beta$  = 0.3,  $\gamma$  = 0.1, name = "0"),
    ( $\beta$  = 0.1,  $\gamma$  = 0.05, name = "1"),
    ( $\beta$  = 1.0,  $\gamma$  = 0.2, name = "2")
]

tmax = 100.0

for exp in experiments
     $\beta$ ,  $\gamma$ , name = exp. $\beta$ , exp. $\gamma$ , exp.name

    println("\nЭксперимент $name:  $\beta$  =  $\beta$ ,  $\gamma$  =  $\gamma$ ,  $R_0$  =  $(\beta/\gamma)$ ")

    # Создаём сеть Петри для SIR модели
    net, u0, states = build_sir_network( $\beta$ ,  $\gamma$ )

    # Детерминированная симуляция (решение ОДУ)
    df_det = simulate_deterministic(
        net, u0, (0.0, tmax),
        saveat = 0.5,
        rates = [ $\beta$ ,  $\gamma$ ]
    )
end

```

Рисунок 4.6: Скрипт базового эксперимента (с параметром)

Для обоих типов симуляции, ничего сильно меняется, просто время пика (рис. 7, 8, 9, 10, 11, 12)

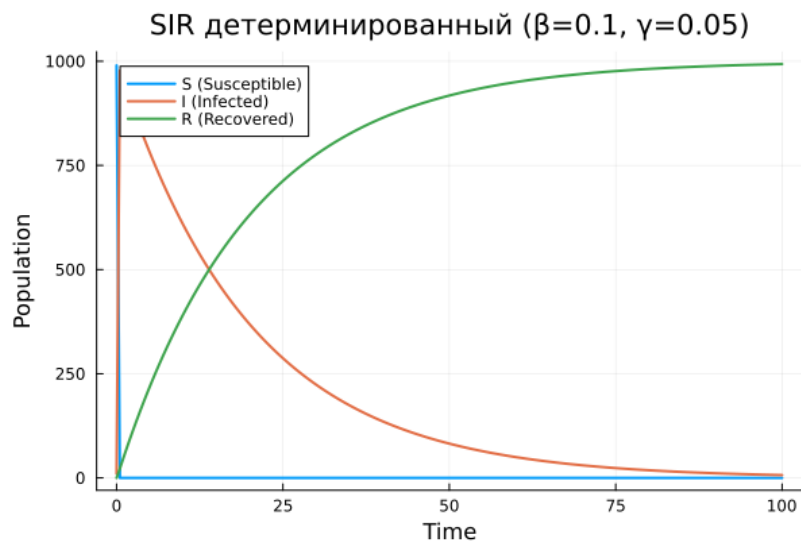


Рисунок 4.7: Детерминистический график динамики SIR

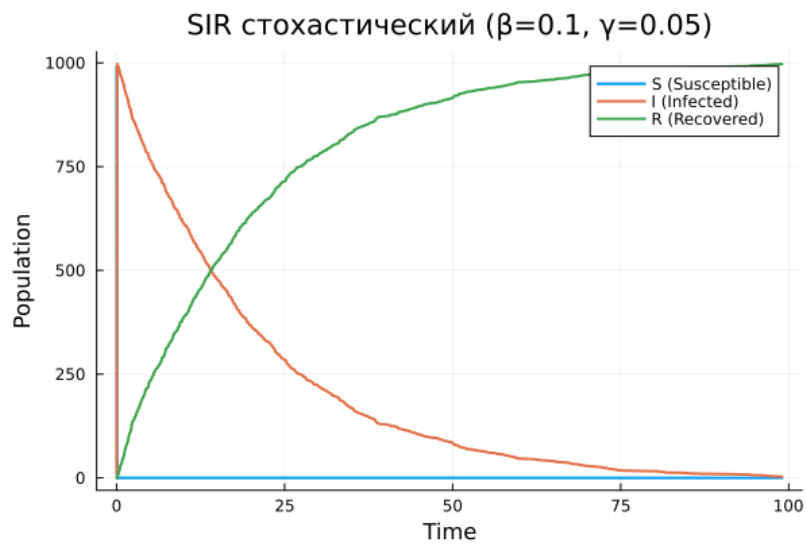


Рисунок 4.8: Стохастический график динамики SIR

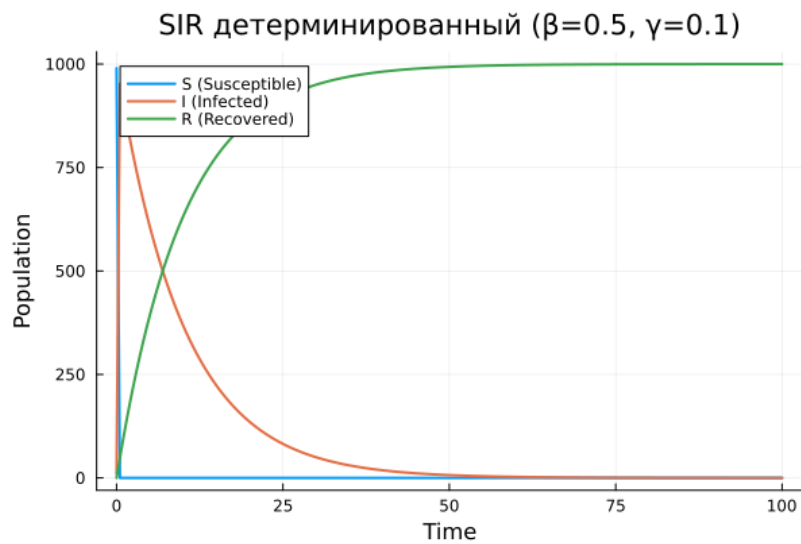


Рисунок 4.9: Детерминистический график динамики SIR

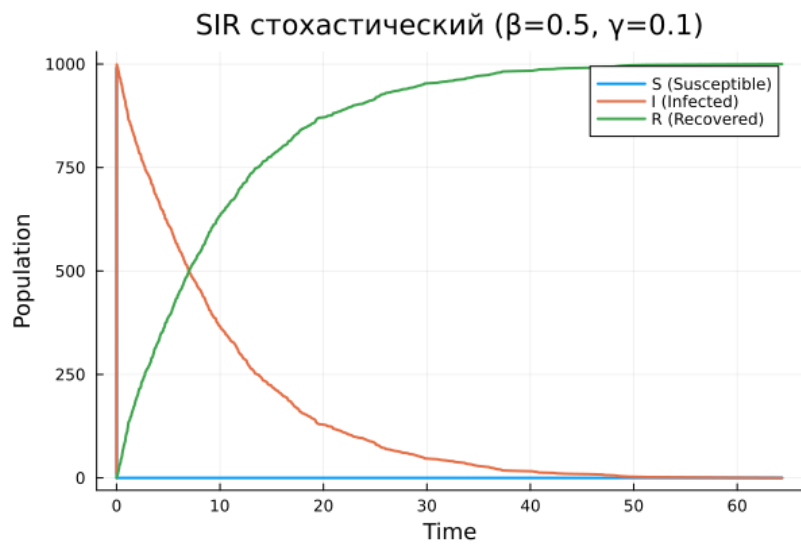


Рисунок 4.10: Стохастический график динамики SIR

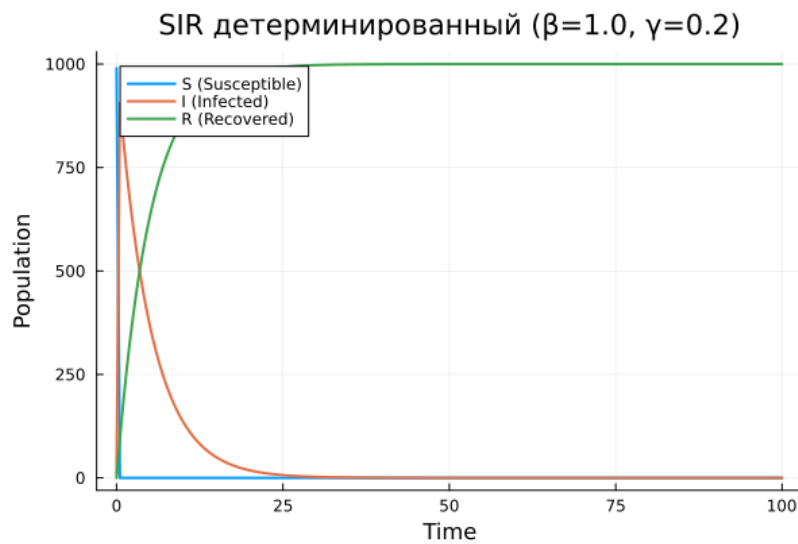


Рисунок 4.11: Детерминистический график динамики SIR

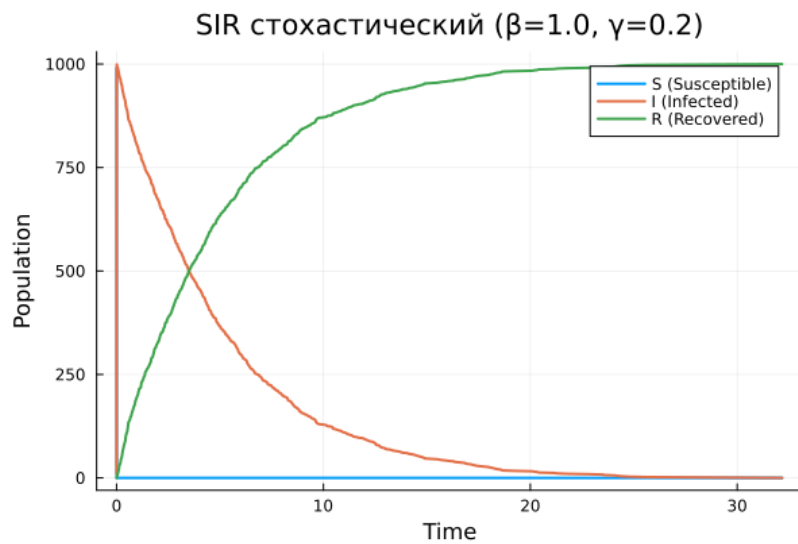


Рисунок 4.12: Стохастический график динамики SIR

Я генерировала файл jupyter, запускала все ячейке и все сработала без ошибок(рис.13)

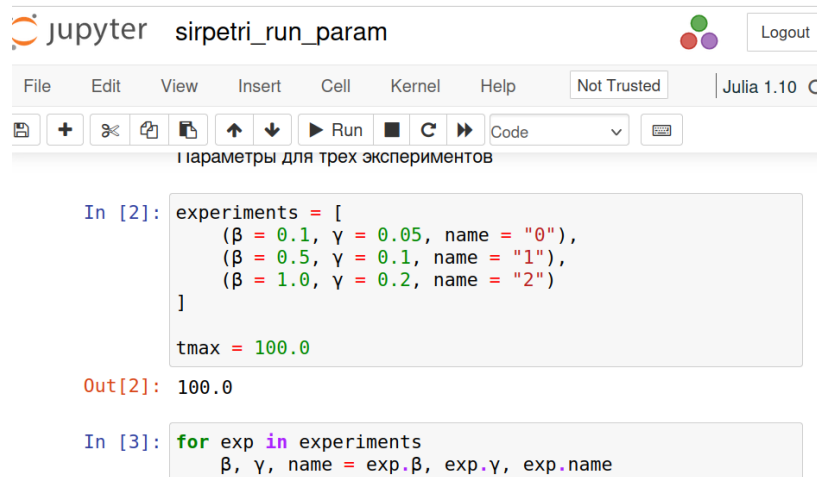


Рисунок 4.13: файл jupyter

Ниже приведен код, результаты расчетов и графики для кода в файле `scripts/sirpetri_run_param.qmd`

4.4 Базовый прогон модели SIR (Petri nets)

```

using DrWatson
@quickactivate "project"
using Random
include(sourcedir("SIRPetri.jl"))
using .SIRPetri
using DataFrames, CSV, Plots

```

```

└ Warning: DrWatson could not find find a project file by recursively checking given `dir
  | (given dir: /home/nelianjovu)
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
WARNING: replacing module SIRPetri.
WARNING: using SIRPetri.build_sir_network in module Main conflicts with an existing iden
WARNING: using SIRPetri.simulate_deterministic in module Main conflicts with an existing

```

WARNING: using SIRPetri.simulate_stochastic in module Main conflicts with an existing identifier.
WARNING: using SIRPetri.plot_sir in module Main conflicts with an existing identifier.

Выполняет базовые эксперименты с разными параметрами β и γ .

Параметры для трёх экспериментов

```
experiments = [  
    ( $\beta$  = 0.1,  $\gamma$  = 0.05, name = "0"),  
    ( $\beta$  = 0.5,  $\gamma$  = 0.1, name = "1"),  
    ( $\beta$  = 1.0,  $\gamma$  = 0.2, name = "2")  
]
```

```
tmax = 100.0
```

```
100.0
```

```
for exp in experiments  
     $\beta$ ,  $\gamma$ , name = exp. $\beta$ , exp. $\gamma$ , exp.name  
    println("\nЭксперимент $name:  $\beta$  =  $\beta$ ,  $\gamma$  =  $\gamma$ ,  $R_0$  =  $(\beta/\gamma)$ ")  
    net, u0, states = build_sir_network( $\beta$ ,  $\gamma$ ) # Создаём сеть Петри для SIR модели  
  
    df_det = simulate_deterministic(  
        net, u0, (0.0, tmax),  
        saveat = 0.5,  
        rates = [ $\beta$ ,  $\gamma$ ]  
    )# Детерминированная симуляция (решение ОДУ)  
  
    Random.seed!(123)  
    df_stoch = simulate_stochastic(  
        net, u0, (0.0, tmax),
```



```

        rates = [0, 0]
    )# Стохастическая симуляция (алгоритм Гиллеспи)

    CSV.write(datadir("sir_det_$name.csv"), df_det)
    CSV.write(datadir("sir_stoch_$name.csv"), df_stoch)

    p_det = plot_sir(df_det) # Строим и сохраняем графики
    title!(p_det, "SIR детерминированный ( $\beta$ =$ $\beta$ ,  $\gamma$ =$ $\gamma$ )")
    savefig(plotsdir("sir_det_dynamics_$name.png"))

    p_stoch = plot_sir(df_stoch)
    title!(p_stoch, "SIR стохастический ( $\beta$ =$ $\beta$ ,  $\gamma$ =$ $\gamma$ )")
    savefig(plotsdir("sir_stoch_dynamics_$name.png"))

end

println("\nБазовый прогон завершён. Результаты в data/ и plots/")

```

Эксперимент 0: $\beta = 0.1$, $\gamma = 0.05$, $R_0 = 2.0$

Эксперимент 1: $\beta = 0.5$, $\gamma = 0.1$, $R_0 = 5.0$

Эксперимент 2: $\beta = 1.0$, $\gamma = 0.2$, $R_0 = 5.0$

Базовый прогон завершён. Результаты в data/ и plots/

4.5 3. Коэффициент заражения β

Я создала файл `scripts/sirpetri_scan_parameters.jl` и скопировала заданный скрипт в него. Скрипт исследует чувствительность модели к изменению параметра β (скорости заражения). Для каждого значения β из диапазона `0.1:0.05:0.8` запускается детерминированная симуляция ($\gamma = 0.1$ фиксирован) и вычисляются: максимальное число инфицированных (пик эпидемии) – `peak_I` и конечное число выздоровевших – `final_R`. Я преобразовала скрипт в литературный стиль и запускала его (рис.14)

```
| sirpetri_scan_parameters.jl ×
# ## Коэффициент заражения  $\beta$ 

using DrWatson
@quickactivate "project"
include(scrdir("SIRPetri.jl"))
using .SIRPetri
using DataFrames, CSV, Plots

# Исследуется чувствительность модели к изменению параметра  $\beta$ 
# (скорости заражения)
 $\beta\_range = 0.1:0.05:0.8$ 
 $\gamma\_fixed = 0.1$ 
 $tmax = 100.0$ 

results = []
# Для каждого значения  $\beta$  из диапазона 0.1:0.05:0.8 запускается
# детерминированная симуляция ( $\gamma = 0.1$  фиксирован)
#
# Для каждого прогона вычисляются:
#
# 1. максимальное число инфицированных (пик эпидемии) – peak_I;
# 2. конечное число выздоровевших – final_R.
```

Рисунок 4.14: Скрипт коэффициента заражения β

После запуск, получила график. На графике показано, что при всех тестируемых значениях β инфекция распространяется почти по всей популяции, что соответствует конечному значению $R \approx N$. В результате в этом моделировании не наблюдается порогового значения. Пиковое число инфицированных индивидуумов изменяется незначительно при изменении β (рис 15)

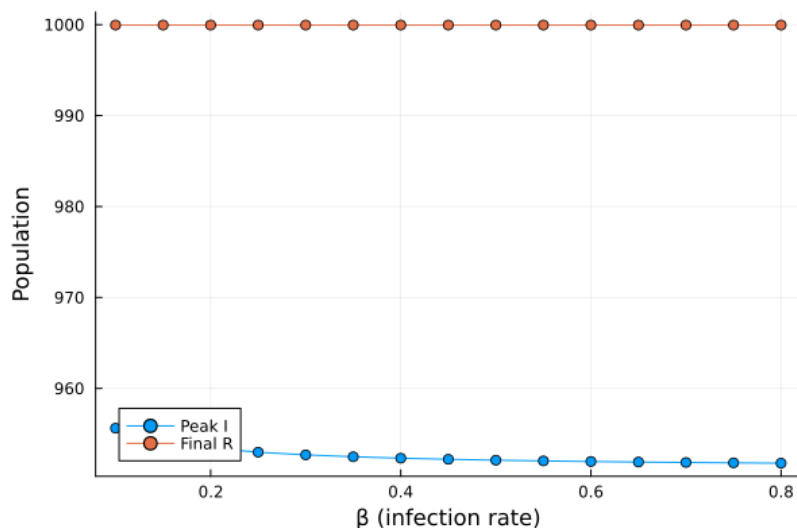
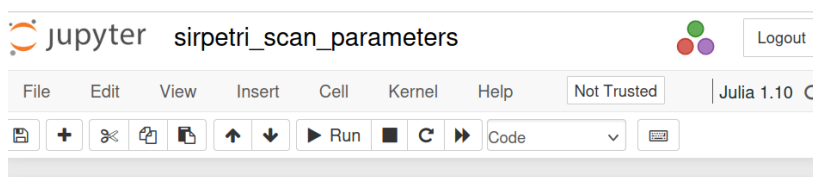


Рисунок 4.15: Коэффициент заражения β

Я генерировала файл jupyter, запускала все ячейке и все сработала без ошибок(рис.16)



Коэффициент заражения β

```
In [1]: using DrWatson
@quickactivate "project"
include(scrdir("SIRPetri.jl"))
using .SIRPetri
using DataFrames, CSV, Plots
```

Исследуется чувствительность модели к изменению параметра β
(скорости заражения)

Рисунок 4.16: файл jupyter

Ниже приведен код, результаты расчетов и графики для кода в файле scripts/sirpetri_scan_parameters.qmd

4.6 Коэффициент заражения β

```
using DrWatson
@quickactivate "project"
include(sourcedir("SIRPetri.jl"))
using .SIRPetri
using DataFrames, CSV, Plots
```

```
└ Warning: DrWatson could not find find a project file by recursively checking given `dir`
  | (given dir: /home/nelianjovu)
  └ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
WARNING: replacing module SIRPetri.
WARNING: using SIRPetri.build_sir_network in module Main conflicts with an existing identifier
WARNING: using SIRPetri.simulate_deterministic in module Main conflicts with an existing identifier
WARNING: using SIRPetri.simulate_stochastic in module Main conflicts with an existing identifier
WARNING: using SIRPetri.plot_sir in module Main conflicts with an existing identifier.
```

Исследуется чувствительность модели к изменению параметра β (скорости заражения)

```
β_range = 0.1:0.05:0.8
β_fixed = 0.1
tmax = 100.0

results = []
```

Any[]

Для каждого значения β из диапазона 0.1:0.05:0.8 запускается детерминированная симуляция ($\gamma = 0.1$ фиксирован)

Для каждого прогона вычисляются:

1. максимальное число инфицированных (пик эпидемии) – peak_I;
2. конечное число выздоровевших – final_R.

```
for β in β_range
    net, u0, _ = build_sir_network(β, β_fixed)
    df = simulate_deterministic(net, u0, (0.0, tmax), saveat = 0.5, rates = [β, β_fixed])
    peak_I = maximum(df.I)
    final_R = df.R[end]
    push!(results, (β = β, peak_I = peak_I, final_R = final_R))
end
```

Результаты сводятся в таблицу и строится график зависимости peak_I(β) и final_R(β)

```
df_scan = DataFrame(results)
CSV.write(datadir("sir_scan.csv"), df_scan)
```

```
"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab06/project/data/sir_scan.csv"
```

График

```
p = plot(
    df_scan.β,
    [df_scan.peak_I df_scan.final_R],
    label = ["Peak I" "Final R"],
    marker = :circle,
    xlabel = "β (infection rate)",
    ylabel = "Population",
)
```

```
savefig(plotsdir("sir_scan.png"))

println("Сканирование □ завершено. Результат в data/sir_scan.csv")
```

Сканирование □ завершено. Результат в data/sir_scan.csv

4.7 4. Анимация детерминированной динамики

Я создала файл scripts/sirpetri_animate.jl и писала скрипт в него. Скрипт создаёт GIF-анимацию, показывающую, как со временем меняется количество людей в каждой из трёх групп (S, I, R). Анимация позволяет наглядно увидеть распространение эпидемии, пик и спад. (рис.17)

```
| sirpetri_animate.jl × | sirpetri_scan_parameters.jl ×
# ## Анимация детерминированной динамики

using DrWatson
@quickactivate "project"

include(sourcedir("SIRPetri.jl"))
using .SIRPetri
using Plots, Random

β = 0.3
γ = 0.1
tmax = 100.0
fps = 5

net, u0, names = build_sir_network(β, γ)
df = simulate_deterministic(net, u0, (0.0, tmax), saveat = 0.5,
rates = [β, γ])
anim = @animate for row in eachrow(df)
    u = [row.S, row.I, row.R]
    bar(
        ["S", "I", "R"],
        u,
        legend = false,
        xlabel = "Compartments",
        ylabel = "Population",
        title = "t = $(round(row.time, digits=1))",
        ylims = (0, maximum(u0) + 5)
    )
end
gif(anim, plotsdir("sir_animation.gif"), fps = fps)
```

Рисунок 4.17: Скрипт анимации

После запуск, получила GIF. В момент времени $t = 0$ популяция почти полно-

стью восприимчива, имеется лишь небольшое число инфицированных и ни одного выздоровевшего человека. Это представляет собой начальную стадию распространения болезни, с которой развивается эпидемия.

4.8 5. Итоговый отчёт

Я создала файл `scripts/sirpetri_report.jl` и скопировала заданный скрипт в него. Скрипт загружает ранее сохранённые результаты (`sir_det.csv`, `sir_stoch.csv`, `sir_scan.csv`) и строит сравнительные графики для итогового отчёта: Сравнение детерминированной и стохастической динамики инфицированных $I(t)$ и зависимость пика I от β (результат сканирования). Я преобразовала скрипт в литературный стиль и запускала его(рис.18)

```
┌ sirpetri_animate.jl × ─┐ sirpetri_report.jl × ─┐
# ## Итоговый отчёт
using DrWatson
@quickactivate "project"
using DataFrames, CSV, Plots

# Загружает ранее сохранённые результаты (sir_det.csv,
sir_stoch.csv, sir_scan.csv) и строит сравнительные графики для
итогового отчёта:
# 1. Сравнение детерминированной и стохастической динамики
инфицированных  $I(t)$ .
# 2. Зависимость пика  $I$  от  $\beta$  (результат сканирования).
df_det = CSV.read(datadir("sir_det.csv"), DataFrame)
df_stoch = CSV.read(datadir("sir_stoch.csv"), DataFrame)
df_scan = CSV.read(datadir("sir_scan.csv"), DataFrame)

# Сравнение детерминированной и стохастической динамики
p1 = plot(
    df_det.time,
    [df_det.I df_stoch.I[1:length(df_det.time)]],
    label = ["Deterministic I" "Stochastic I"],
    xlabel = "Time",
    ylabel = "Infected",
    title = "Comparison",
)
savefig(plotsdir("comparison.png"))

# Зависимость пика  $I$  от  $\beta$ 
p2 = plot(
    df_scan.β,
    df_scan.peak_I,
```

Рисунок 4.18: Скрипт итогового отчёта

После запуск, получила два графики. Сравнение детерминированной и

стохастической кривых $I(t)$ показывает, насколько велик разброс, вызванный случайностью. При большом размере популяции они близки. График чувствительности позволяет количественно оценить, как изменение заразности β влияет на тяжесть эпидемии (пик I). Это важно для принятия решений (например, меры по снижению β)(рис 19 и 20)

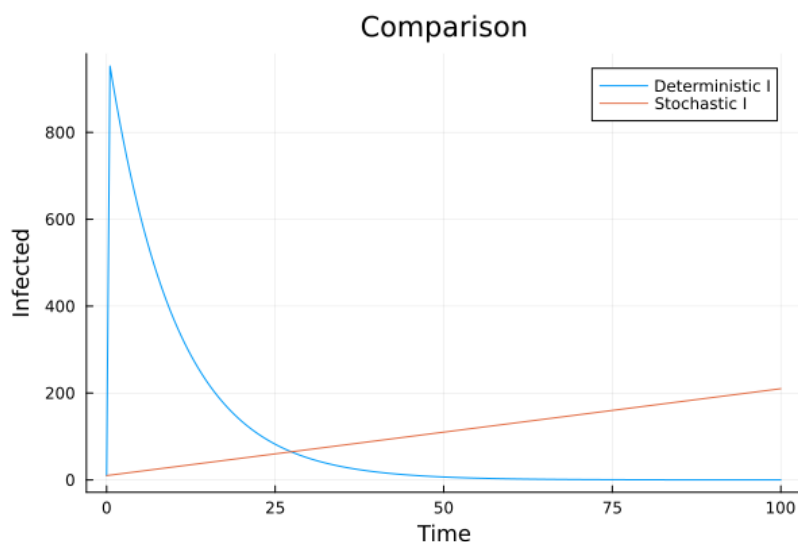


Рисунок 4.19: Детерминированный и стохастический графики

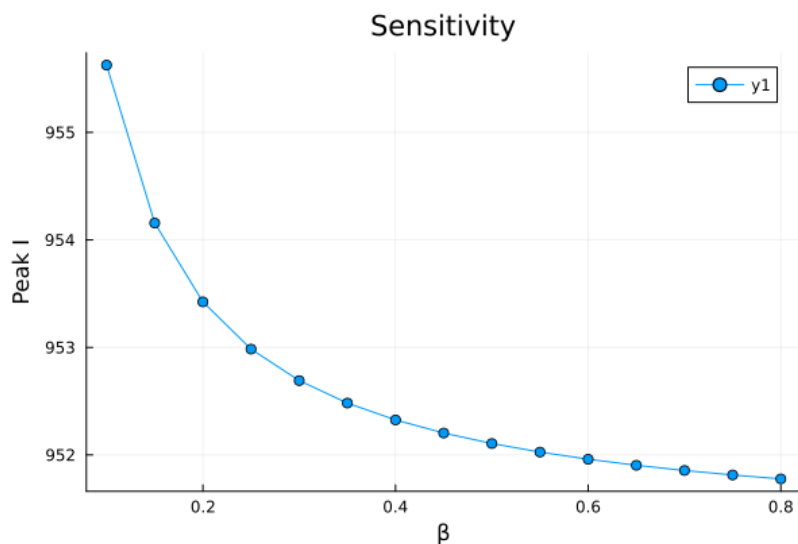


Рисунок 4.20: Зависимость пика I от β

Я генерировала файл jupyter, запускала все ячейке и все сработала без оши-

бок(рис.21)

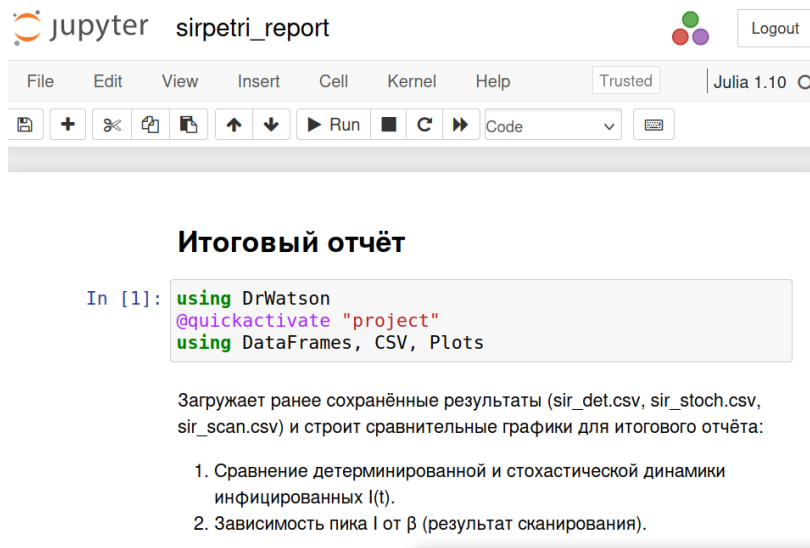


Рисунок 4.21: файл jupyter

Ниже приведен код, результаты расчетов и графики для кода в файле scripts/sirpetri_report.qmd

4.9 Итоговый отчёт

```
using DrWatson
@quickactivate "project"
using DataFrames, CSV, Plots
```

```
└ Warning: DrWatson could not find find a project file by recursively checking given `dir`
  | (given dir: /home/nelianjovu)
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

Загружает ранее сохранённые результаты (sir_det.csv, sir_stoch.csv, sir_scan.csv) и строит сравнительные графики для итогового отчёта: 1.

Сравнение детерминированной и стохастической динамики инфицированных $I(t)$. 2. Зависимость пика I от β (результат сканирования).

```
df_det = CSV.read(datadir("sir_det.csv"), DataFrame)
df_stoch = CSV.read(datadir("sir_stoch.csv"), DataFrame)
df_scan = CSV.read(datadir("sir_scan.csv"), DataFrame)
```

	β	peak_I	final_R
	Float64	Float64	Float64
1	0.1	955.626	999.954
2	0.15	954.157	999.954
3	0.2	953.424	999.954
4	0.25	952.984	999.955
5	0.3	952.691	999.955
6	0.35	952.482	999.955
7	0.4	952.326	999.955
8	0.45	952.204	999.955
9	0.5	952.106	999.955
10	0.55	952.026	999.955
11	0.6	951.96	999.955
12	0.65	951.904	999.955
13	0.7	951.856	999.955
14	0.75	951.814	999.955
15	0.8	951.777	999.955

Сравнение детерминированной и стохастической динамики

```
p1 = plot(
    df_det.time,
    [df_det.I df_stoch.I[1:length(df_det.time)]],
    label = ["Deterministic I" "Stochastic I"],
```

```

    xlabel = "Time",
    ylabel = "Infected",
    title = "Comparison",
)
savefig(plotsdir("comparison.png"))

```

"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab06/project/plots/comparison.png"

Зависимость пика I от β

```

p2 = plot(
    df_scan.□,
    df_scan.peak_I,
    marker = :circle,
    xlabel = "□",
    ylabel = "Peak I",
    title = "Sensitivity",
)
savefig(plotsdir("sensitivity.png"))

println("Отчётные графики сохранены в plots/")

```

Отчётные графики сохранены в plots/

5 Выводы

В этой лабораторной работы, реализовали основные модели(модель SIR) в подходе сетей Петри.